1 Real-time engine

The real-time path reuses the estimator, measurement model and state manager of post-processing and reorganises the control flow around a *streaming* contract. Complete RTCM 3 (or SBF) frames arrive one at a time from a file or a network socket and are decoded into observations, corrections and broadcast ephemeris. A buffered epoch passes through the Kalman-filter predict/update cycle as soon as its boundary is detected. Post-processing iterates a closed list of epochs in time order with the full orbit and clock product set on disk; the engine sees each epoch once, receives corrections that may lead or lag the observations, and cannot look ahead. The entry point is `Application::runRealTime` (`src/core/app/ApplicationRealtime.cpp`), which constructs a `RealTimeEngine` (`src/core/realtime/RealTimeEngine.cpp`) and drives it from a single-threaded multi-source poll loop. The engine takes its input through the frame feeds `feedRtcmFrame` and `feedSbfFrame`; at stream end `flush` processes whatever epochs are still buffered. This section covers the parts specific to that path. The estimation mathematics, the observation equations and the SSR correction model are documented elsewhere and are only cross-referenced here.

1.1 Streaming control flow

The poll loop holds one `RealtimeStream` per logical source and services the sources in turn without blocking on any one of them:

- a *primary* stream carrying rover observations (RTCM 3 MSM or Septentrio SBF, from a file replay, an NTRIP mountpoint, or a raw TCP socket);
- an optional *corrections* stream (typically an IGS-RTS NTRIP mountpoint carrying SSR);
- an optional *ephemeris* stream (broadcast nav for an obs-only station that does not co-broadcast 1019/1020/...).

Each iteration drains a bounded burst from every active source through its framer and dispatches the complete frames to the engine (`pumpFrames`). The primary drain is capped at 1 MiB per iteration, and the corrections and ephemeris streams are read one 4 KiB chunk per iteration, so a high-rate observation stream cannot starve the sparse SSR and navigation streams. When a live network-product tail is configured, the loop also polls it at about 5 Hz and wakes the obs/SSR hold buffer of Section 1.7 when new products arrive. The loop exits when every source is inactive (file EOF or client stopped), when a stop is requested (Ctrl+C or the console-close handler), or when the `processing.max_duration_s` data-time budget latches. On exit the engine flushes the final buffered epoch before teardown, so no observations or trace rows are lost.

A socket source counts as *active* while its client is running, whether or not it is connected at that moment. A transient drop or the pre-connect window at startup therefore does not end the session; the client reconnects in the background.

1.2 NTRIP transport

Caster connection. `NtripClient` (`src/core/realtime/NtripClient.cpp`) is an NTRIP v2 client built on libcurl. A background thread holds the HTTP GET open and appends the received bytes to a mutex-guarded buffer with a hard cap of 4 MiB. When the cap is reached the buffer drops its oldest bytes. Consumers read through a blocking `read` or a timeout-bounded `tryRead`;

the multi-stream loop uses the latter, so one stream cannot block the others. The request carries the NTRIP v2 headers (`Ntrip-Version:Ntrip/2.0` and a `User-Agent`), and the mountpoint is the URL path. HTTP Basic authentication (`CURLAUTH_BASIC`) is supplied when a username is configured. An optional NMEA GGA sentence for VRS mountpoints is sent as the NTRIP v2 `Ntrip-GGA` request header on the GET; the request is never turned into a POST.

Transport security. The transport defaults to plain HTTP, which serves plain-HTTP casters such as IGS-IP (`igs-ip.net`) unchanged. A `use_tls` flag selects the `https` URL scheme and sets `CURLUSESSL_ALL`, as required for port-443 casters such as CDDIS that offer no plain-HTTP endpoint. Certificate verification (`SSL_VERIFYPEER/VERIFYHOST`) stays on unless a testing-only `tls_insecure` override disables it for self-signed or mis-chained casters. Both flags default off, so a plain-HTTP run is unaffected.

Reconnection. The receive loop reconnects automatically. After each transfer ends it classifies and logs the outcome: a transport error, a non-200 HTTP status (with hints for 401, 403 and 404), a connection that delivered zero bytes (an idle mountpoint, or a VRS mountpoint that needs a GGA position), or a clean stream end. It then always waits `reconnect_delay_s` plus a uniform 0–2000 ms jitter before retrying, so a misconfigured mountpoint appears in the log as an error and several streams dropped by one caster outage do not reconnect in lockstep. An NTRIP v1 caster answers "ICY 200 OK", which libcurl reports as status code 0; the loop treats code 0 as success.

1.3 RTCM 3 framing and CRC

`RtcmStreamReader` (`src/core/realtime/RtcmStreamReader.cpp`) turns an arbitrary byte stream into discrete RTCM 3 frames and so separates transport from parsing. An RTCM 3 frame is

$$\underbrace{\texttt{0xD3}}_{\text{preamble}}\ \underbrace{[\,6\text{ reserved}\,|\,10\text{-bit length }L\,]}_{2\text{ bytes}}\ \underbrace{[\,\text{payload, }L\text{ bytes}\,]}\ \underbrace{[\,\text{CRC-24Q}\,]}_{3\text{ bytes}}, \tag{1}$$

so the total frame length is $3 + L + 3$. `nextFrame` scans for the `0xD3` preamble (discarding other bytes), reads the 10-bit payload length (a length above 1023 bytes is rejected as a false preamble), waits until the full $3 + L + 3$ bytes are buffered, and verifies the trailing 24-bit Qualcomm CRC (CRC-24Q, table-driven) computed over header+payload. On a CRC mismatch the reader advances past the candidate preamble byte and resumes scanning, so a corrupt or misframed byte cannot stall it. Only CRC-valid frames are emitted, and the downstream parser (`RtcmParser`) relies on frame integrity. The reader counts extracted frames and discarded bytes for stream-health diagnostics.

Septentrio SBF. A parallel binary path decodes Septentrio SBF frames into the same observation and ephemeris outputs (`src/core/sbf/`). `SbfStreamReader` performs the `$@` (`0x24 0x40`) sync and the CRC-16/XMODEM check, and `SbfParser` dispatches blocks by ID and decodes the `MeasEpoch` observations. `Meas3Decoder` unpacks the compact, differentially encoded `Meas3Ranges` observations; GLONASS FDMA slots are resolved through `SbfSignalMap`. `SubframeAssembler` rebuilds broadcast ephemeris from raw-nav subframes. This section covers only the framing and the feed seam (Sections 1.1 and 1.9). The `Meas3Ranges` differential range reconstruction is the direct analogue of (2) and (3) below and the one part of the SBF path that is algebra rather than ICD bookkeeping. It is *not* yet derived in this reference, and no other document derives it: this is a known gap. The remaining SBF detail (`SubframeAssembler` ICD scaling, the `SbfSignalMap` tables, the framing constants) belongs in the Doxygen headers and is out of scope here.

The top-level `RtcmParser` (`src/core/rtcm/RtcmParser.cpp`) reads the 12-bit message type DF002 from the first 12 bits of the payload (the frame header carries only the length) and dispatches on it: 1005/1006 station coordinates; 1033 receiver and antenna descriptors and 1230 GLONASS code-phase biases (both logged, neither consumed); 4076 IGS-SSR; legacy SSR (1057–1068, 1240–1270); broadcast ephemeris (1019/1020/1042/1044/1046); and MSM4, MSM5 and MSM7 observations. The week that promotes a truncated epoch field to a full `GnssTime` comes from the parser's per-stream time tracker rather than from the host clock at each frame (Section 1.6). One parser instance serves the primary, corrections and ephemeris streams, so all three share one GPS timeline.

1.4 MSM observation decode

A single body (`decodeMsmImpl`, `src/core/rtcm/RtcmMsmDecoder.cpp`) decodes every MSM observation message. It is parameterised over an `MsmVariant` descriptor that supplies the per-field bit widths, scale factors, sentinels and capability flags. Three variants are instantiated: MSM4, MSM5 and MSM7. The variant mechanism can describe MSM6, but no MSM6 entry exists, so MSM6 is not decoded. The header carries a 64-bit satellite mask, a 32-bit signal mask and an $N_{\text{sat}} \times N_{\text{sig}}$ cell mask; the number of cells is $N_{\text{cell}} = \text{popcount(cell mask)}$. Each field block is packed contiguously across all satellites or all cells, in mask order, and is decoded into parallel vectors before recombination.

Range reconstruction. For each satellite the rough range in milliseconds is the sum of the 8-bit integer part (DF397) and the 10-bit modulo part (2^{-10} ms, DF398). Each cell adds a signed fine term with a variant-specific scale. With c the speed of light, the pseudorange and carrier phase of a cell are

$$P = c \cdot 10^{-3}\Big(n_{\text{int}} + n_{\text{mod}}\, 2^{-10} + n^{P}_{\text{fine}}\, \sigma_P\Big), \tag{2}$$

$$\Phi_{[\text{m}]} = c \cdot 10^{-3}\Big(n_{\text{int}} + n_{\text{mod}}\, 2^{-10} + n^{\Phi}_{\text{fine}}\, \sigma_\Phi\Big), \qquad \Phi_{[\text{cyc}]} = \Phi_{[\text{m}]}/\lambda, \tag{3}$$

with fine scales $\sigma_P = 2^{-24}$, $\sigma_\Phi = 2^{-29}$ for MSM4 (and MSM5) and $\sigma_P = 2^{-29}$, $\sigma_\Phi = 2^{-31}$ for MSM7 (all in ms). Cells holding the RTCM-defined invalid sentinel are skipped. Carrier phase is emitted in cycles, the RINEX-3 convention the observation model consumes (the observation-equations section); when the wavelength λ cannot be resolved, the phase is stored in metres under the same observation code. The decoder also produces CNR (DF403 in whole dB-Hz for MSM4/5, DF408 in 1/16 dB-Hz for MSM7) and, for MSM5/7, a Doppler value from the rough+fine phase-range-rate (the negated rate over the wavelength). The half-cycle-ambiguity bit (DF420) sets the LLI half-cycle flag.

DF419 extended-satellite-info gating. The 4-bit DF419 "extended satellite information" field determines the framing of the per-satellite block. In MSM5 and MSM7 it is present in the bitstream for every constellation, but only GLONASS assigns it a meaning (the FDMA frequency-channel number). The decoder therefore gates two operations separately:

- *bitstream consumption* is gated on the MSM variant (`honour_ext_sat_info`, true for MSM5/MSM7, false for MSM4/MSM6): when set, $4\,N_{\text{sat}}$ bits are read for all systems;
- *interpretation* is gated separately on the system descriptor; only GLONASS maps the value to a channel.

Reading DF419 only for GLONASS (or skipping it in MSM7) under-consumes the per-satellite block by $4\,N_{\mathrm{sat}}$ bits, so the per-cell fine pseudoranges are read from the wrong bit offset. The rough range survives but the fine terms are biased, which produces an L1/L2 split of order hundreds of metres with the wrong sign on the geometry-free combination. These observations look plausible and pass an over-read check. Gating on the variant is the fix for the GPS L1/L2 geometry-free sign anomaly, and it matches RTCM 10403.3 and the cssrlib reference. As a second guard, the parser's `warnIfUnderConsumed` warns whenever a decode leaves ≥ 8 payload bits unread, which detects this class of misalignment where the bit reader's over-read flag does not.

1.5 Broadcast nav, SSR and station coordinates

Broadcast ephemeris. Messages 1019 (GPS LNAV), 1044 (QZSS), 1046 (Galileo I/NAV), 1042 (BeiDou D1) and 1020 (GLONASS) are decoded by `RtcmNavDecoder` into a common `BroadcastEphemeris` and pushed into the process-wide `LiveEphemerisStore`, from which the geometry layer resolves satellite orbits when no SP3/CLK file is loaded. The field order differs by message (QZSS, for example, leads with its clock block). GLONASS carries a sign-magnitude PZ-90 state vector, which is integrated separately.

SSR corrections. `RtcmSsrDecoder` decodes both the legacy SSR messages and the IGS-SSR 4076 container and pushes the records into the shared `SsrCorrectionStore` with their SSR header (provider epoch and IOD-SSR), so staleness and IOD diagnostics refer to the provider solution rather than to the time of receipt. The store holds orbit, clock, combined, code-bias, phase-bias, URA and high-rate-clock records per satellite. The orbit and clock query paths read the store directly, so no per-epoch wiring is needed. The correction mathematics (the radial/along/cross orbit projection, the clock and bias application, the ionosphere model) is documented in the SSR-corrections section and is not repeated here. Two reactions to the stream are specific to the real-time path:

- an incremented IGS-SSR phase-bias *discontinuity counter* for a signal breaks the integer property of that signal's ambiguity. By default the engine resets the affected ambiguity before the same epoch's measurement update (`StateManager::noteSsrDiscontinuity` detects the counter change). Two opt-in knobs replace the reset: the declared-step carry `processing.ssr_phase_bias_disc_carry` (card CC) and the step translation `processing.ssr_phase_bias_step_translate` (card PBT, which also covers undeclared steps and takes precedence). Both shift the ambiguity by the phase-bias step instead of re-floating it;
- every SSR push re-attempts a held observation epoch (Section 1.7), because the correction horizon has just advanced.

Station coordinates (1005/1006). `decodeStationCoords` reads the antenna reference point (ARP) as three signed 38-bit ECEF components in 0.1 mm units; 1006 adds a 16-bit antenna height in the same unit. A reference station that streams its own coordinates thereby supplies an a-priori seed (Section 1.9).

1.6 Time-system handling

Streaming messages carry truncated, system-specific epoch fields, which are promoted to a common GPS-time `GnssTime`. Each parser instance keeps a tracker, in continuous GPS seconds, of the last decoded stream epoch. It promotes every mod-week or time-of-day field by *nearest continuation* to that tracker: the week (or day) that minimises $|\Delta t|$, as in RTKLIB's `adjweek` (`RtcmParser::`

anchorGpsSeconds). On this path the host clock (TimeUtils::nowContinuousGpsSeconds, POSIX UTC plus the leap offset) is read only at cold start, on the first time-bearing frame after construction, so a skewed host clock or a race at a week rollover cannot move an established stream to another week. Each successful MSM or SSR epoch advances the tracker (updateTimeAnchor). The Keplerian broadcast-nav decoders read the tracker for their week but do not advance it. The anchor below is the tracker's current value. One exception: the 1020 GLONASS ephemeris TOE is anchored on the host clock at every message (see the GLONASS item below).

- **Flat ms-of-week systems** (GPS/GAL/QZS/SBAS/BDS MSM): the 30-bit epoch is milliseconds of week. BeiDou adds the constant 14 s BDT→GPS offset and re-wraps the result into the week.
- **GLONASS** carries its 30-bit MSM epoch as a 3-bit day-of-week followed by a 27-bit time-of-day in ms. Reading only 27 bits misaligns the satellite mask and corrupts the whole GLONASS decode. TimeUtils::glonassTodToGpsTime promotes the ms-of-day: it places the reported time-of-day in the GLONASS = UTC(SU)+3 h ("Moscow") day nearest the stream-time tracker and converts back to GPS time, re-adding the leap seconds:

$$t_{\mathrm{GPS}} = \left(t^{0}_{\mathrm{day}} + \tilde{t}_{\mathrm{TOD}} \right) - \Delta_{\mathrm{Moscow}} + \Delta_{\mathrm{leap}}, \tag{4}$$

  where t^{0}_{day} is the Moscow-day midnight nearest the anchor and $\tilde{t}_{\mathrm{TOD}}$ is the reported time-of-day placed into that day by resolving the midnight wrap toward the nearest match (± 12 h). The 3-bit day-of-week is read but not used, since the anchor resolves the day. The same conversion promotes any GLONASS SSR epoch against the tracker. It also promotes the 1020 ephemeris toe, but there the anchor is the host clock, read at each message, rather than the stream tracker. On a live stream the two agree to well within 12 h; on a file replay of historical data they do not.
- **GLONASS FDMA wavelength.** GLONASS L1/L2 are frequency-division: each slot transmits on $f = f_{\mathrm{base}} + k\,\Delta f$ with channel $k \in [-7, +6]$, so the carrier wavelength is per satellite. The decoder maps DF419 to the channel as $k = v - 7$ for the valid range and uses it in Equation (3). The channel is also stored on the observation, so the slot map and the MW combinations can resolve frequency. When DF419 is absent (MSM4/MSM6) or holds a reserved value, the channel is marked unknown and the phase is stored in metres; a default of zero would decode to a false $k = -7$.

Each decoded record carries its own content epoch (epoch_time). The synchronisation logic of Section 1.7 compares against that epoch, not against the time of arrival.

1.7 Observation/SSR synchronisation

PPP-RTK needs observations time-*aligned* with the corrections rather than with the host clock. In a live session the observation stream commonly runs ahead of a lagging SSR stream. An observation epoch that no satellite's SSR can cover is processed entirely at broadcast grade (and, with drop_broadcast_grade, loses every satellite). The opt-in processing.enable_obs_ssr_sync hold buffer prevents this. With t_{obs} the observation epoch, the SSR coverage horizon is the freshest stored SSR content epoch (SsrCorrectionStore::newestSsrEpoch) plus the maximum SSR age:

$$t_{\mathrm{horizon}} = t^{\mathrm{newest}}_{\mathrm{SSR}} + \tau_{\max}, \qquad \tau_{\max} = \texttt{ssr_max_age_s}. \tag{5}$$

The decision is:

- $t_{\mathrm{obs}} \le t_{\mathrm{horizon}}$: SSR covers the epoch. The epoch is processed normally and any hold is cleared;

- $t_{\text{obs}} > t_{\text{horizon}}$: the epoch is *deferred*. The hold buffer keeps only the latest such epoch (skip-to-latest semantics), and `drainHeldObs` re-attempts it as soon as an SSR push (or a new network-product epoch, through `notifyProductsAdvanced`) advances the horizon.

A liveness backstop keeps a dead SSR feed from starving the filter. If a hold persists past `obs_ssr_sync_max_hold_s` of host-clock time, the engine latches a degraded "outage" state and passes every epoch through (at broadcast grade, or dropped by the existing path) until SSR covers the observations again. The held epoch is drained with skip-to-latest *bypassed*, so the SSR-aligned epoch is not discarded. The final `flush` bypasses both the hold and skip-to-latest, so the last epoch is always processed. The feature is inert when disabled, before the first epoch, and in any run without a live SSR orbit and clock source.

A separate, coarser `enable_obs_skip_to_latest` drops observation epochs whose lag behind the *host-clock* GPS time exceeds `obs_skip_max_lag_s` (default 5 s), so a backlogged stream fast-forwards to the freshest epoch. The skip returns before any continuity state is touched, so the next processed epoch measures its time step from the last accepted one and trips the gap path (Section 1.10) when the skipped span exceeds the gap tolerance.

1.8 Ephemeris management

`LiveEphemerisStore` keeps, per satellite, the current record and one previous record (the current record is demoted to previous when the IODE changes), and evaluates the orbit and clock against the appropriate one. Two policies govern it in real time.

- **Freshest-toe selection.** An incoming record is rejected when its `toe`, expressed in continuous GPS seconds (`toeGpsContinuous`, which reconciles the BDT and GLONASS time scales), is older than the held record's by more than 1 s. A stale or re-ordered broadcast therefore never overwrites a newer one. A record from the IGS stream is also protected from a receiver-decoded record at equal `toe`: the IGS-RTS SSR deltas are computed against the IGS broadcast, and a receiver ephemeris with different fit parameters would make the pairing inconsistent. Receiver nav replaces an IGS-stream record only when its reference epoch is at least 1800 s newer (the GLONASS nominal `toe` step), which indicates that the IGS stream has stalled on that satellite.
- **toe-aware and IOD-aware selection.** For orbit and clock evaluation the store picks the record whose `toe` is nearest the query epoch (`pick`). For an SSR-corrected satellite it first selects the record the SSR delta was computed for (`pickForSsr`, which matches the SSR IODE against the current and the retained previous record). Across an ephemeris cutover the receiver can be one IOD ahead of the SSR stream; without this re-reference the IOD mismatch would drop the satellite to broadcast grade.

1.9 A-priori seeding from RTCM 1005/1006

A *live* SBF/RTCM observation stream carries no RINEX `APPROX POSITION` record (a file-fed source does; see the third item below). Without an a-priori position the engine falls back to a coarse Earth-centred seed at the geographic pole. `seedInitialState` calls `PppModel::initializeState` and then applies the fallback, since a state write before that call would be overwritten. The model it calls is built by `initProcessingComponents`, so `seedInitialState` must follow it in the construction order. From there almost every satellite sits below the elevation mask, the first `computeModels` yields no usable satellite, WLS never runs and the filter cannot bootstrap. The engine therefore seeds the a-priori position from the stream itself until the Kalman filter owns the position:

- from a *PVTGeodetic* solution in an SBF stream (`feedSbfFrame`);
- from an RTCM *1005/1006* station ARP (`feedRtcmFrame`), the RTCM analogue of the PVT seed. This makes `receiver.reference_position` optional for any station that streams 1005/1006. An optional blunder gate (`realtime.arp_blunder_gate_m`, RTA-12(3); 0, the default, disables it) rejects a broadcast ARP that disagrees with an existing a-priori by more than that distance, so a wrong mountpoint or stale 1005 coordinates cannot move the seed;
- from the RINEX header of a *file-fed* source, reduced from the marker to the antenna reference point through `ANTENNA:DELTA H/E/N` (`RinexParser::getArpPositionXyz`, which calls `Coordinates::markerToArp`). The position state, and the ATX PCO/PCV applied to it, refer to the ARP rather than the monument, so seeding on `APPROX POSITION XYZ` unreduced would start the filter one eccentricity away. The reduction is an exact no-op for a zero or absent eccentricity, so a live SBF/RTCM stream is unaffected; it matters only for the file-fed closed-loop consumer, whose header carries a real eccentricity. The reduction is not applied twice: `PppModel::initializeState` reduces from the *raw* marker, and only while the state position is still zero or non-finite.

A configured `reference_position` always takes precedence. The seed is applied only while `first_epoch_` is set. The startup gate keeps that flag set through warmup, so a 1005/1006 message that arrives a few seconds into the stream (these messages are sent less often than the 1 Hz observations) still seeds before the first estimation epoch. The flag clears in the first epoch that passes the startup gate, after measurement conditioning and before the predict (Section 1.10). From then on the filter owns the position, and a decoded ARP or PVT is only logged for comparison.

1.10 The per-epoch real-time update

`processEpoch` runs the predict/update pipeline of post-processing, with streaming-specific guards in front and the post-processing-only stages removed. It is a thin ordered driver. Each step is a private member function (a *seam*) in one of the files `src/core/realtime/RealTimeEngine*.cpp`, grouped by concern: admission (`RealTimeEngineAdmit.cpp`), measurement-set preparation (`RealTimeEnginePrepare.cpp`), slip QC (`RealTimeEngineSlipQc.cpp`), SSR datum-break reactions (`RealTimeEngineSsrBreaks.cpp`) and estimation (`RealTimeEngineEstimate.cpp`). Seams exchange per-epoch values through `RtEpochContext`, which holds nothing beyond the epoch; a value that must survive to the next epoch is a member of the engine. The four admission gates are the only seams that may end an epoch early. The driver itself also ends an epoch when no observation of an active system remains or when `computeModels` fails; after the startup gate every step runs unconditionally. An optional staged filter-state snapshot (warm resume) is restored before the gates, so the first time step spans the actual outage. In order:

1. **Lag, sync and duration gates.** The host-clock backlog warning, the obs↔SSR hold of Section 1.7 and the skip-to-latest drop form the first gate (`admitBacklogAndSync`); the `max_duration_s` latch is the second (`gateMaxDuration`). Both run before any continuity state is touched.
2. **Step and gap detection** (the third gate, `stepTimeAndDetectGap`). $\Delta t = t_k - t_{k-1}$, with a week-rollover correction; the first epoch assumes 1 s. A non-advancing $\Delta t \leq 0$ is a duplicate (a second co-timestamped SBF block or a retransmit) and is skipped with the state untouched. Folding it in would double-count the measurements and pass $\Delta t = 0$ to the process-noise build, which throws. A Δt above the gap tolerance (`processing.data_gap_tolerance_s`, 1 s when unset) flags a gap.

3. **Models and startup gate** (the fourth gate, `gateCorrectionsReady`). Observations are filtered to the active systems and run through `computeModels`. When `defer_until_corrections_ready` is armed, estimation (no predict, no update) is deferred until enough cm-grade broadcast+SSR satellites are present: 5 in total and 4 per active constellation by default, the per-constellation floor reduced to what the constellation currently fields but never below two. A timeout forces a degraded start. The filter then initialises code and phase together on a clean state instead of cold-starting on metre-grade broadcast clocks.
4. **Measurement-set conditioning.** After two observability sweeps (the SSR coverage log and the SSR consumer trace), `dropBroadcastGrade` removes broadcast-grade satellites when the startup gate's `drop_broadcast_grade` is set. In modes with a receiver-clock state, `seedClockFromWls` replaces the filter clock with a WLS clock estimate when the two differ by at least 5×10^{-5} s (`kJumpThresh`, 0.05 ms, about 15 km), after a gap, or on the first epoch. Under BSSD (no receiver-clock state) `regaugeBssd` subtracts the median of the epoch's code innovations, one receiver clock common to all satellites, from every code and phase observable. This common-mode shift leaves the single differences exactly unchanged and keeps the per-satellite bias states at DCB scale instead of absorbing the full unsteered receiver clock. The clock seed and the BSSD re-gauge are mutually exclusive: the first requires a receiver-clock state, the second its absence. The first-epoch flag (`first_epoch_`) clears at the end of this step.
5. **Predict.** `predictAided` over $(t_{k-1}, t_k]$, the aiding path shared with post-processing; for a GNSS-only stream it delegates byte-identically to `predict`(Δt) of the estimation-engine section.
6. **Cycle-slip detection and repair** (`detectAndRepairSlips`). The MW detector, the optional TurboEdit MW+GF joint repair and the soft-LLI gap bridge run before the update, with the detectors and repair machinery of the cycle-slip section. They are enabled under `estimation.cycle_slip`. TurboEdit is for high-rate data only; its GF-arc gap tolerance is 1.5 times the configured stream rate (1 Hz if none is set), because a live stream has no RINEX interval header.
7. **SSR datum-break reactions.** These react to the producer moving its datum and do nothing without SSR content. The order is fixed. A change of the SSR solution-generation id (`reactToGenerationChange`) resets every ambiguity and re-inflates the receiver code-bias states; it runs first because it subsumes the per-signal reactions. `logProducerStatus` logs a producer status transition. `resetOnPhaseBiasDisc` resets (or, with cards CC or PBT, shifts) the ambiguity of a signal whose phase bias shows a discontinuity or a freshness flip, as described for the SSR corrections above. `reinflateOnCodeBiasDisc` re-inflates the receiver code-bias states on a code-bias datum re-reference, and `reinflateOnClockDisc` re-inflates the affected inter-system bias states on a clock datum re-reference. Last, the opt-in declared-level-step reaction `reactToDeclaredLevelSteps_` (card CLV2, `processing.ssr_level_step_inflate`, default off) compares each satellite's producer-declared clock and code-bias levels with their previous values. When the resulting step in a code signal's level exceeds a 0.1 mm dead band, it inflates the covariance along the direction that moves the modelled code but not the modelled phase (the satellite's slant ionosphere together with its ambiguities, each ambiguity weighted by the squared ratio of the GPS L1 frequency to its signal frequency). The added standard deviation is the step size times a configured scale. A satellite without an ionosphere state or an ambiguity is refused.
8. **Measurement update** (`runMeasurementUpdate`). The outlier and forced-pivot-handover rebuild loop of the post-processing driver (shared via `EpochUpdateSupport`): rebuild on `NeedsMeasurementRebuild`, forward each constellation's forced pivot handover, re-inflate reset-satellite states, clear the MW/TurboEdit arc history of KF-reset signals, and remove one outlier per pass. Phase signals held by the soft-LLI bridge are excluded from the update.

9. **Pseudo-observation constraints.** After the update and before AR, so the reduced covariance reaches the resolver. An optional soft a-priori ZWD pseudo-observation (`applyZwdConstraint`, `estimation.zwd_constraint`) anchors the BSSD [up, ZWD] near-null subspace. When the engine consumes a network ionosphere product, `applyNetworkIonoPrior` constrains each satellite's ionosphere state with a slant-ionosphere prior from the same epoch, with an error block common to each constellation.
10. **Ambiguity resolution** (`resolveAmbiguities`). After a successful float update, MLAMBDA AR runs on the committed state with the configuration surface of post-processing and is fed the MW-resolved wide-lane integers (the ambiguity-resolution section). The closed-loop network-bias validation suppresses the wide-lane integers, so AR runs on the float state alone.
11. **State extraction and trace** (`writeEpochTraces`). The posterior is synced back to the PPP state and the trace rows are written. The optional ambiguity-level check (card ALC), the per-satellite consumer readout (card CST) and the periodic filter-state snapshot follow.

Post-processing stages skipped in real time. Non-causal and batch stages do not fit a streaming pipeline without look-ahead and are absent:

- the backward RTS smoother is disabled under `AppMode::RealTime`, because smoothing requires the full forward trajectory and cannot run causally;
- geometry relinearisation (the IEKF second pass) is a post-processing-only refinement and is not replicated;
- precise SP3/CLK products are not loaded when a live broadcast+SSR source replaces them (a real-time *file* replay without corrections still loads them). Orbit and clock are gated separately, so a configuration with precise orbits and network clocks loads the SP3 file and skips the CLK file;
- the stages that need a GIM have no source in the stream: higher-order ionosphere, GIM ionosphere weighting and adaptive ionospheric process noise are skipped with a warning when enabled.

1.11 Assessing a real-time run

A streaming run has no answer key, unlike a simulated one. The self-referenced statistic of the ordinary state viewer (scatter about the run's own median) cannot see an error *common* to every epoch, and a correction-driven pipeline is most likely to make that kind of error. A static receiver on a surveyed mark supplies the missing datum. Let $\boldsymbol{r}_{\mathrm{ref}}$ be the surveyed antenna position (`receiver.reference_position_ecef`) with geodetic latitude and longitude (φ, λ), and $\hat{\boldsymbol{r}}_k$ the filter's posterior position at epoch k. The error is the ECEF difference rotated into the local horizon frame at the *reference* point,

$$\begin{bmatrix} e_k \\ n_k \\ u_k \end{bmatrix} = \boldsymbol{R}(\varphi, \lambda)\left(\hat{\boldsymbol{r}}_k - \boldsymbol{r}_{\mathrm{ref}}\right), \qquad \boldsymbol{R} = \begin{bmatrix} -\sin\lambda & \cos\lambda & 0 \\ -\sin\varphi\cos\lambda & -\sin\varphi\sin\lambda & \cos\varphi \\ \cos\varphi\cos\lambda & \cos\varphi\sin\lambda & \sin\varphi \end{bmatrix}, \tag{6}$$

with $\varepsilon_k^{\mathrm{3D}} = \|(e_k, n_k, u_k)\|$ and $\varepsilon_k^{\mathrm{H}} = \|(e_k, n_k)\|$. The difference is taken in ECEF before the rotation, so the rotation origin cancels and (6) is exact rather than a local-tangent approximation.

Convergence must be suffix-stable. The first epoch whose error falls below a threshold measures the noise of a solution, not the solution. A float PPP-RTK trajectory dips under any

loose bound long before it *holds* it, and a run that later diverges scores the same as one that does not. The convergence epoch for threshold θ is therefore defined as

$$k^\star(\theta) = \min\Big\{\, k : \max_{j \geq k} \; \varepsilon_j^{\mathrm{3D}} \leq \theta \,\Big\}, \tag{7}$$

the first epoch from which the error stays within θ for the *entire remainder* of the run ($k^\star = \varnothing$ when no such epoch exists, the correct verdict for a run that never settles). The suffix maximum is computed in one reverse cumulative pass. The suffix function counts a non-finite error as a violation, but the scorecard first drops every trace row whose position is non-finite, so an epoch without a position is skipped rather than breaking the hold. The metric is reported as the ladder $\theta \in \{1.0, 0.5, 0.2, 0.1\}$ m rather than a single number, which keeps it informative for runs of very different quality. The steady-state statistics are taken over $k \geq k^\star$ of the coarsest achieved rung, so the convergence transient is excluded from the accuracy claim by construction; when no rung is held, they fall back to the second half of the run and the summary flags the fallback. The window is stated alongside the number it produced.

Covariance credibility. Grading the solution against the surveyed point also tests whether the filter's own uncertainty is credible. With $\boldsymbol{P}_k^{xx}$ the position block of the posterior covariance, the rotation-invariant scalar

$$\sigma_k^{\mathrm{3D}} = \sqrt{\operatorname{tr} \boldsymbol{P}_k^{xx}} = \sqrt{\sigma_x^2 + \sigma_y^2 + \sigma_z^2} \tag{8}$$

is compared directly with $\varepsilon_k^{\mathrm{3D}}$. The trace is invariant under the rotation (6), so no per-axis horizon σ has to be constructed from a trace that carries only the diagonal. A float solution whose error persistently exceeds $3\sigma^{\mathrm{3D}}$ is *overconfident* as well as inaccurate. This is the usual signature of unmodelled correction error absorbed by ambiguity and bias states whose formal variances do not reflect it. The simulator dashboard applies the same covariance-optimism diagnostic against its answer key; here the surveyed mark takes the place of the truth trajectory.

What the reference does not certify. The surveyed point grades *position* only. A static run cannot separate a receiver-clock error from a common-mode range error, and under BSSD (Section 1.10) there is no receiver-clock state to grade. The per-satellite bias states are gauge-fixed, so their absolute values cannot be checked against an external reference either. A position that grades well therefore certifies the geometry and the correction stack *jointly*, not any single component. The real-time scorecard viewer (`pyviz/gnssviz/viewers/rt_scorecard.py`) of the report built by `pyviz/run_report.py --run-config` implements (6)–(8). An explicit `--ref-ecef` overrides the configured reference position.